

Sec 1 รหัสนักศึกษา 67050050 ชื่อ สกุล ขรินทรัพย์ เอี่ยมสร้อย

Mac Address ของเครื่อง C8-4B-D6-60-14-59

IP Address ของเครื่องนักศึกษา 10.45.7.218

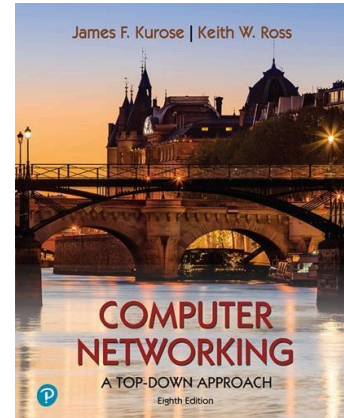
Lab นี้มาจาก Wireshark Lab version 8.0 Page 1 of 12

Wireshark Lab: TCP v8.1

Supplement to *Computer Networking: A Top-Down Approach*, 8th ed., J.F. Kurose and K.W. Ross

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved



In this lab, we'll investigate the behavior of the celebrated TCP protocol in detail. We'll do so by analyzing a trace of the TCP segments sent and received in transferring a 150KB file (containing the text of Lewis Carrol's *Alice's Adventures in Wonderland*) from your computer to a remote server. We'll study TCP's use of sequence and acknowledgement numbers for providing reliable data transfer; we'll see TCP's congestion control algorithm – slow start and congestion avoidance – in action; and we'll look at TCP's receiver-advertised flow control mechanism. We'll also briefly consider TCP connection setup and we'll investigate the performance (throughput and round-trip time) of the TCP connection between your computer and the server.

Before beginning this lab, you'll probably want to review sections 3.5 and 3.7 in the text¹.

1. Capturing a bulk TCP transfer from your computer to a remote server

Before beginning our exploration of TCP, we'll need to use Wireshark to obtain a packet trace of the TCP transfer of a file from your computer to a remote server. You'll do so by accessing a Web page that will allow you to enter the name of a file stored on your

¹ References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach*, 8th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020. Our website for this book is http://gaia.cs.umass.edu/kurose_ross You'll find lots of interesting open material there.

computer (which contains the ASCII text of *Alice in Wonderland*), and then transfer the file to a Web server using the HTTP POST method (see section 2.2.3 in the text). We're using the POST method rather than the GET method as we'd like to transfer a large amount of data *from* your computer to another computer. Of course, we'll be running Wireshark during this time to obtain the trace of the TCP segments sent and received from your computer.

Do the following:

- Start up your web browser. Go the <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and retrieve an ASCII copy of *Alice in Wonderland*. Store this as a .txt file somewhere on your computer.
- Next go to <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
- You should see a screen that looks like Figure 1.

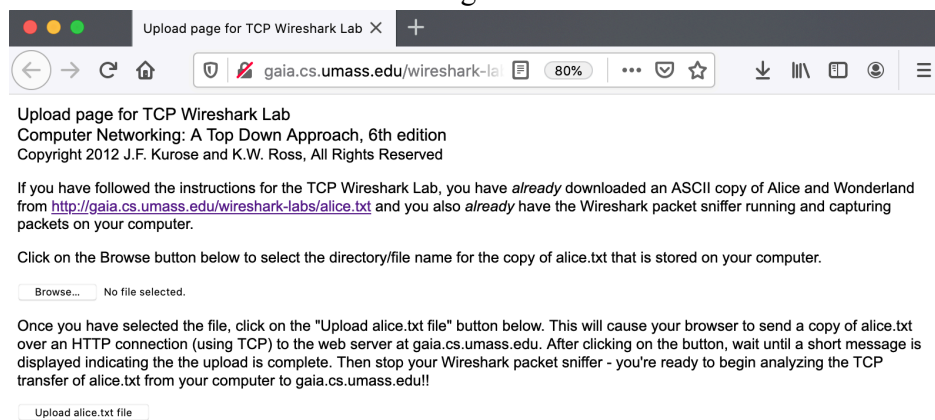


Figure 1: Page to upload the alice.txt file from your computer to gaia.cs.umass.edu

- Use the *Browse* button in this form to the file on your computer that you just created containing *Alice in Wonderland*. Don't press the "*Upload alice.txt file*" button yet.
- Now start up Wireshark and begin packet capture (see the earlier Wireshark labs if you need a refresher on how to do this).
- Returning to your browser, press the "*Upload alice.txt file*" button to upload the file to the gaia.cs.umass.edu server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
- Stop Wireshark packet capture. Your Wireshark window should look similar to the window shown in Figure 2.

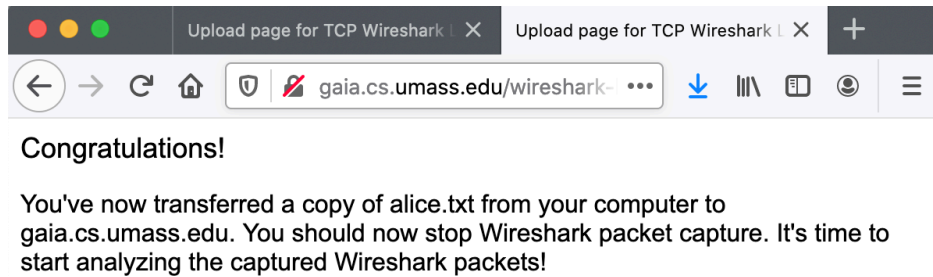


Figure 2: Success! You've uploaded a file to `gaia.cs.umass.edu` and have hopefully captured a Wireshark packet trace while doing so.

If you are unable to run Wireshark on a live network connection, you can download a packet trace that was captured while following the steps above on one of the author's computers². In addition, you may well find it valuable to download this trace even if you've captured your own trace and use it, as well as your own trace, when you explore the questions below.

2. A first look at the captured trace

Before analyzing the behavior of the TCP connection in detail, let's take a high-level view of the trace.

Let's start by looking at the HTTP POST message that uploaded the `alice.txt` file to `gaia.cs.umass.edu` from your computer. Find that file in your Wireshark trace, and expand the HTTP message so we can take a look at the HTTP POST message more carefully. Your Wireshark screen should look something like Figure 3.

² You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file `tcp-wireshark-trace1-1`. This trace file can be used to answer this Wireshark lab without actually capturing packets on your own. This trace was made using Wireshark running on one of the author's computers, while performing the steps indicated in this Wireshark lab. Once you've downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name.

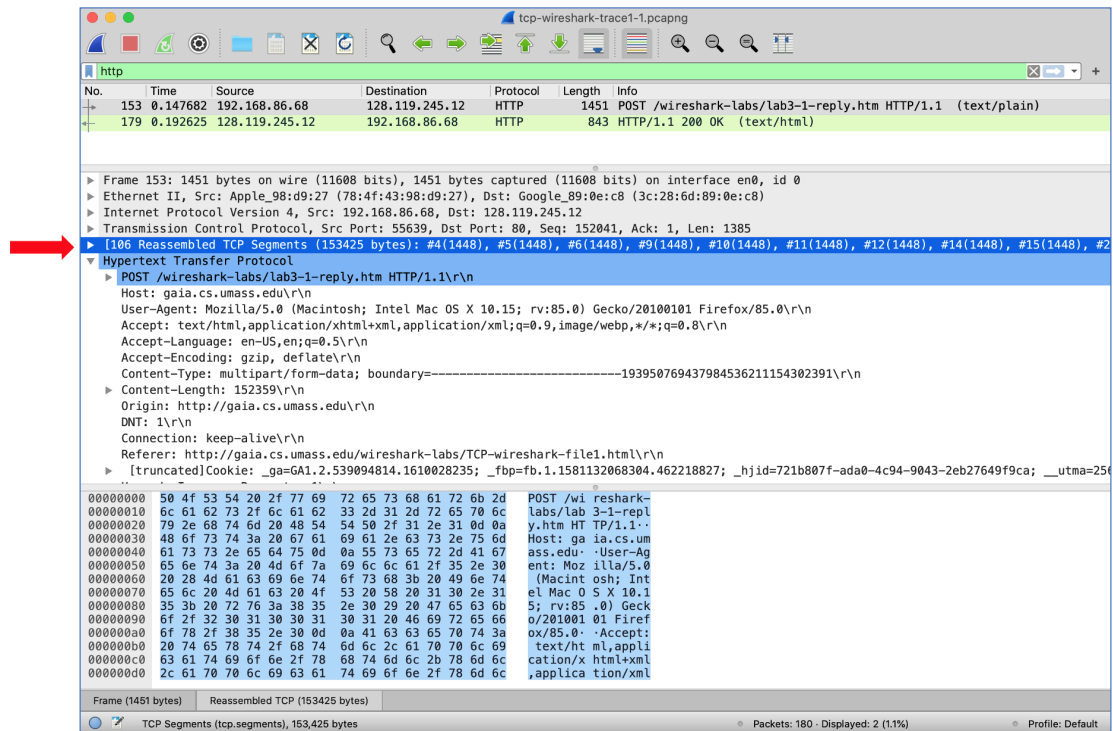


Figure 3: expanding the HTTP POST message that uploaded `alice.txt` from your computer to `gaia.cs.umass.edu`

There are a few things to note here:

- The body of your application-layer HTTP POST message contains the contents of the file `alice.txt`, which is a large file of more than 152K bytes. OK – it’s not *that* large, but it’s going to be too large for this one HTTP POST message to be contained in just one TCP segment!
- In fact, as shown in the Wireshark window in Figure 3 we see that the HTTP POST message was spread across 106 TCP segments. This is shown where the red arrow is placed in Figure 3 [Aside: Wireshark doesn’t have a red arrow like that; we added it to the figure to be helpful ☺]. If you look even more carefully there, you can see that Wireshark is being really helpful to you as well, telling you that the first TCP segment containing the beginning of the POST message is packet #4 in the particular trace for the example in Figure 3, which is the trace `tcp-wireshark-trace1-1` noted in footnote 2. The second TCP segment containing the POST message in packet #5 in the trace, and so on.

Let’s now “get our hands dirty” by looking at some TCP segments.

- First, filter the packets displayed in the Wireshark window by entering “tcp” (lowercase, no quotes, and don’t forget to press return after entering!) into the display filter specification window towards the top of the Wireshark window. Your Wireshark display should look something like Figure 4. In Figure 4, we’ve

noted the TCP segment that has its SYN bit set – this is the first TCP message in the three-way handshake that sets up the TCP connection to `gaia.cs.umass.edu` that will eventually carry the HTTP POST message and the `alice.txt` file. We’ve also noted the SYNACK segment (the second step in TCP three-way handshake), as well as the TCP segment (packet #4, as discussed above) that carries the POST message and the beginning of the `alice.txt` file. Of course, if you’re taking your own trace file, the packet numbers will be different, but you should see similar behavior to that shown in Figures 3 and 4.

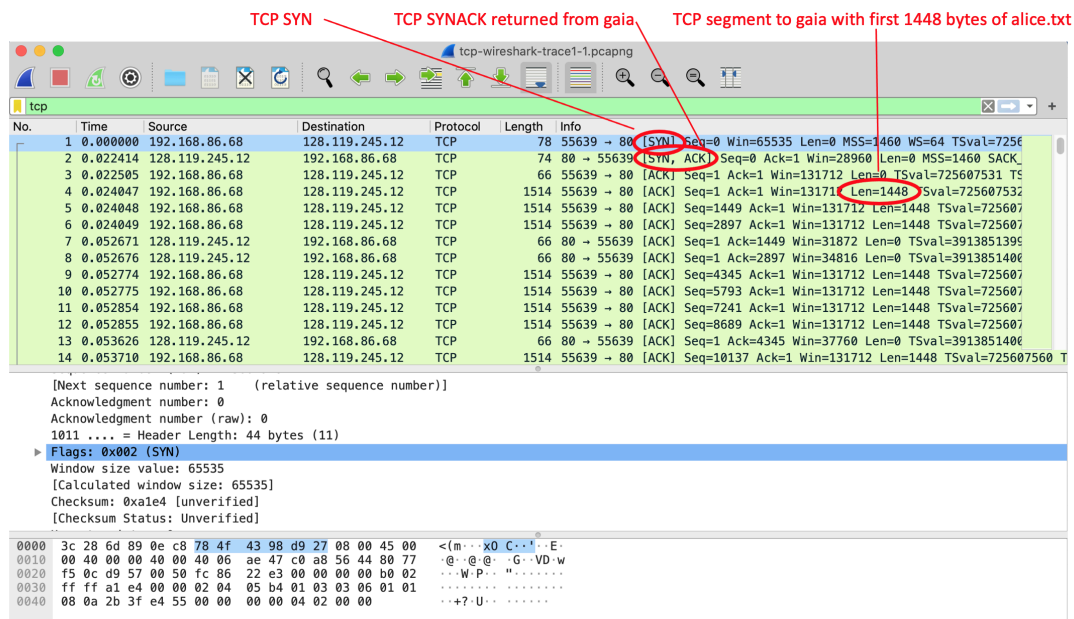


Figure 4: TCP segments involved in sending the HTTP POST message (including the file `alice.txt`) to `gaia.cs.umass.edu`

Answer the following questions³, either from your own live trace, or by opening the Wireshark captured packet file `tcp-wireshark-trace1-1` in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip>

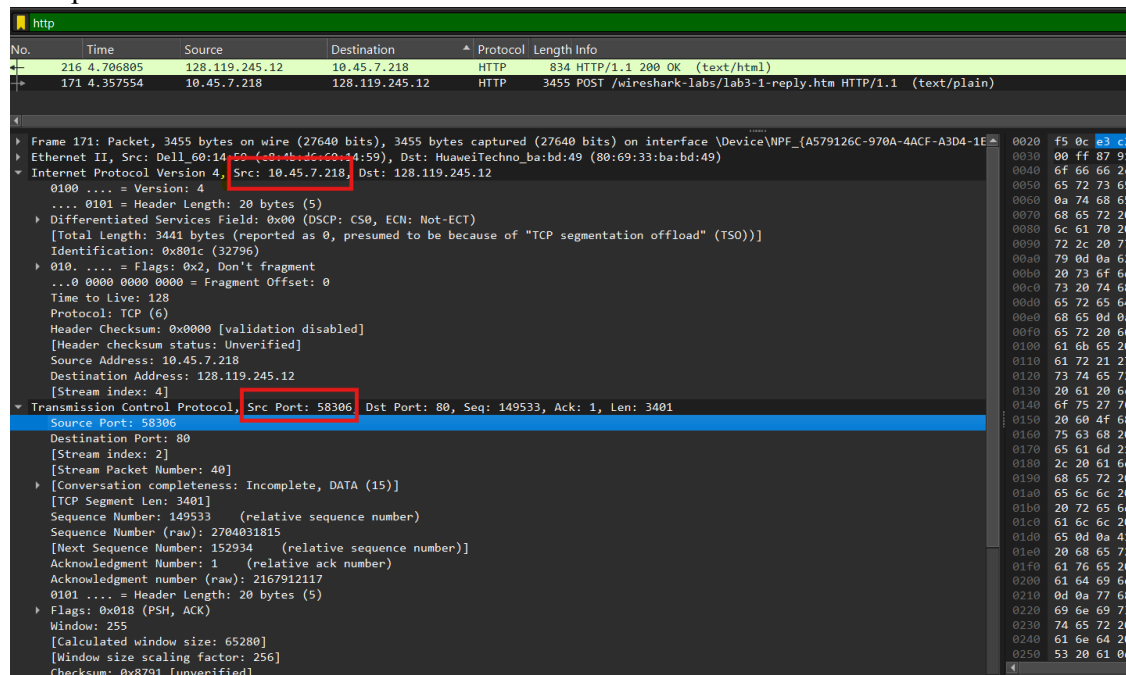
³ For the author’s class, when answering the following questions with hand-in assignments, students sometimes need to print out specific packets (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the packet they’ve found the information that answers a question. They do this by marking paper copies with a pen or annotating electronic copies with text in a colored font. There are also learning management system (LMS) modules for teachers that allow students to answer these questions online and have answers auto-graded for these Wireshark labs at http://gaia.cs.umass.edu/kurose_ross/lms.htm

1. What is the IP address and TCP port number used by the client computer (source) that is transferring the alice.txt file to gaia.cs.umass.edu? To answer this question, it's probably easiest to select an HTTP message and explore the details of the TCP packet used to carry this HTTP message, using the "details of the selected packet header window" (refer to Figure 2 in the "Getting Started with Wireshark" Lab if you're uncertain about the Wireshark windows).

Answer here:

IP address : 10.45.7.210

TCP port : 58306

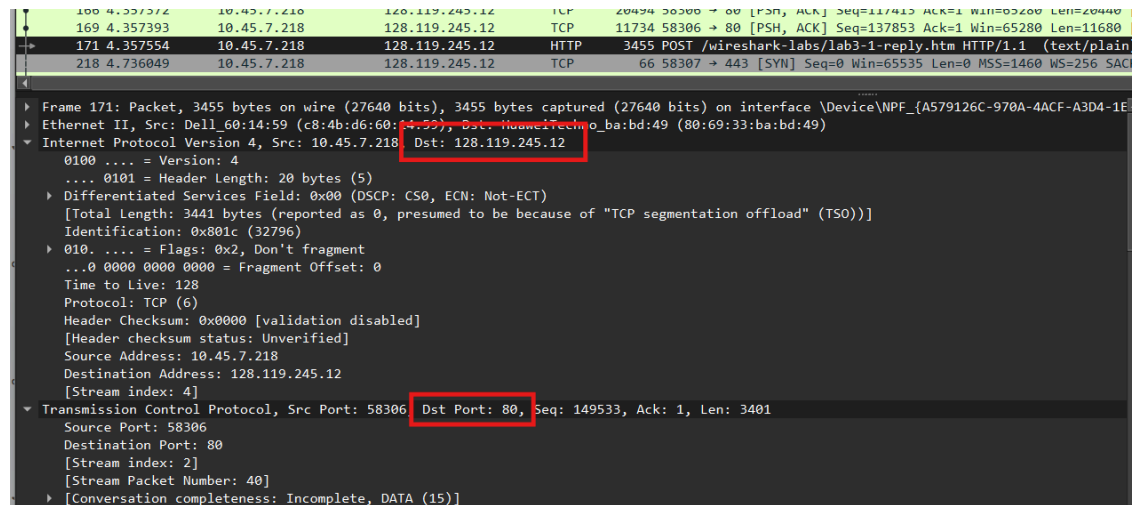


2. What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?

Answer here:

IP address : 128.119.245.12

Port : 80



3. TCP Basics

Answer the following questions for the TCP segments:

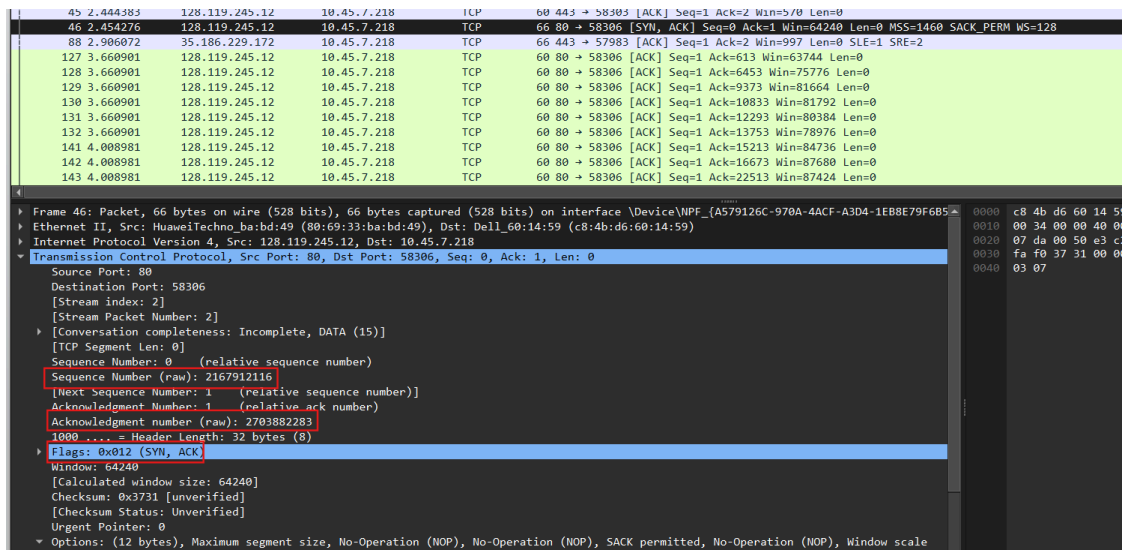
- What is the *sequence number* of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? (Note: this is the “raw” sequence number carried in the TCP segment itself; it is *NOT* the packet # in the “No.” column in the Wireshark window. Remember there is no such thing as a “packet number” in TCP or UDP; as you know, there *are* sequence numbers in TCP and that’s what we’re after here. Also note that this is not the relative sequence number with respect to the starting sequence number of this TCP session.). What is it in this TCP segment that identifies the segment as a SYN segment? Will the TCP receiver in this session be able to use Selective Acknowledgments (allowing TCP to function a bit more like a “selective repeat” receiver, see section 3.4.5 in the text)?

Answer here:

- Sequence Number (raw) = 2703882282
- Segment นี้เป็น SYN เพราะใน TCP Flags มีค่า SYN = 1 (0x002)
- TCP receiver รองรับ Selective Acknowledgment (SACK) เพราะใน TCP Options ระบุว่า SACK permitted

- Answer here:

- Sequence Number (raw) = 2167912116
- Segment นี้เป็น SYNACK เพราะ TCP Flags มีค่า SYN = 1 และ ACK = 1 (0x012)
- Acknowledgement Number (raw) = 2703882283
- ค่า ACK นี้ได้มาจาก
Client's SYN sequence number + 1
(2703882282 + 1 = 2703882283)



5. What is the sequence number of the TCP segment containing the header of the HTTP POST command? Note that in order to find the POST message header, you'll need to dig into the packet content field at the bottom of the Wireshark window, *looking for a segment with the ASCII text "POST" within its DATA field*^{4,5}. How many bytes of data are contained in the payload (data) field of this TCP segment? Did all of the data in the transferred file *alice.txt* fit into this single segment?

Answer here:

- TCP segment ที่มี HTTP POST header คือ Packet No. 171
- Sequence Number (raw) = 2704031815
- Payload (TCP Segment Length) = 3401 bytes
- ข้อมูลไฟล์ *alice.txt* ไม่สามารถอยู่ใน TCP segment เดียวได้ เพราะไฟล์มีขนาด 152,321 bytes และถูกแบ่งส่งเป็นหลาย TCP segments (reassembled หลาย packet)

⁴ Hint: this TCP segment is sent by the client soon (but not always immediately) after the SYNACK segment is received from the server.

⁵ Note that if you filter to only show "http" messages, you'll see that the TCP segment that Wireshark associates with the HTTP POST message is the *last* TCP segment in the connection (which contains the text at the *end* of *alice.txt*: "THE END") and *not* the first data-carrying segment in the connection. Students (and teachers!) often find this unexpected and/or confusing.

```

169 4.357595 10.45.7.218 128.119.245.12 TCP 1154 58306 → 80 [PSH, ACK] Seq=157853 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled]
171 4.357554 10.45.7.218 128.119.245.12 HTTP 3455 POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1 (text/plain)
218 4.736049 10.45.7.218 128.119.245.12 TCP 66 58307 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
220 4.750059 10.45.7.218 128.119.245.12 TCP 54 58306 → 80 [ACK] Seq=152934 Ack=781 Win=64512 Len=0
227 5.002428 10.45.7.218 128.119.245.12 TCP 66 58308 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Source Port: 58306
Destination Port: 80
[Stream index: 2]
[Stream Packet Number: 40]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 3401]
Sequence Number: 149533 (relative sequence number)
Sequence Number (raw): 2704031815
[Next Sequence Number: 152934 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 2167912117
0101 .... = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0x8791 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (3401 bytes)
TCP segment data (3401 bytes)
[11 Reassembled TCP Segments (152933 bytes): #115(612), #116(13140), #133(27740), #146(29200), #149(11680), #152(11680), #154(2920), #162
Hypertext Transfer Protocol
MIME Multipart Media Encapsulation, Type: multipart/form-data, Boundary: "----WebKitFormBoundaryprOwMafXibvSXrhj"
[Type: multipart/form-data]
First boundary: ----WebKitFormBoundaryprOwMafXibvSXrhj\r\n
Packet (3455 bytes)

```

6. What is the length (header plus payload) of each of the first four data-carrying TCP segments?⁶

Answer here:

- Segment 1 (Frame 115) - 612 + 20 = 632 bytes
- Segment 2 (Frame 116) - 13,140 + 20 = 13,160 bytes
- Segment 3 (Frame 133) - 27,740 + 20 = 27,760 bytes
- Segment 4 (Frame 146) - 29,200 + 20 = 29,220 bytes

```

47 2.050009 10.45.7.218 128.119.245.12 TCP 66 58306 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
115 3.312070 10.45.7.218 128.119.245.12 TCP 666 58306 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=612 [TCP PDU reassembled in 171]
116 3.312263 10.45.7.218 128.119.245.12 TCP 13194 58306 → 80 [ACK] Seq=613 Ack=1 Win=65280 Len=13140 [TCP PDU reassembled in 171]
133 3.661016 10.45.7.218 128.119.245.12 TCP 27794 58306 → 80 [PSH, ACK] Seq=13753 Ack=1 Win=65280 Len=27740 [TCP PDU reassembled in 171]
146 4.009082 10.45.7.218 128.119.245.12 TCP 29254 58306 → 80 [PSH, ACK] Seq=41493 Ack=1 Win=65280 Len=29200 [TCP PDU reassembled in 171]
149 4.009388 10.45.7.218 128.119.245.12 TCP 11734 58306 → 80 [ACK] Seq=70693 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
152 4.010052 10.45.7.218 128.119.245.12 TCP 11734 58306 → 80 [PSH, ACK] Seq=82373 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
154 4.010026 10.45.7.218 128.119.245.12 TCP 2974 58306 → 80 [ACK] Seq=94053 Ack=1 Win=65280 Len=2920 [TCP PDU reassembled in 171]
162 4.357201 10.45.7.218 128.119.245.12 TCP 20494 58306 → 80 [PSH, ACK] Seq=96973 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]
166 4.357372 10.45.7.218 128.119.245.12 TCP 20494 58306 → 80 [PSH, ACK] Seq=117413 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]

Frame 115: Packet, 666 bytes on wire (5328 bits), 666 bytes captured (5328 bits) on interface \Device\NPF_{A579126C-970A-4ACF-A3D4-1EBBE7...}
Ethernet II, Src: Dell_60:14:59 (c8:4b:d6:60:14:59), Dst: HuaweiTechno_ba:bd:49 (80:69:33:ba:bd:49)
Internet Protocol Version 4, Src: 10.45.7.218, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 58306, Dst Port: 80, Seq: 1, Ack: 1, Len: 612
Source Port: 58306
Destination Port: 80
[Stream index: 2]
[Stream Packet Number: 4]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 612]
Sequence Number: 1 (relative sequence number)

```

⁶ The TCP segments in the tcp-wireshark-trace1-1 trace file are all less than 1480 bytes. This is because the computer on which the trace was gathered has an interface card that limits the length of the maximum IP datagram to 1500 bytes, and there is a *minimum* of 40 bytes of TCP/IP header data. This 1500-byte value is a fairly typical maximum length for an Internet IP datagram.

115	3.312870	10.45.7.218	128.119.245.12	TCP	666	58306 → 80	[PSH, ACK] Seq=1 Ack=1 Win=65280 Len=612 [TCP PDU reassembled in 171]
116	3.312263	10.45.7.218	128.119.245.12	TCP	13194	58306 → 80	[ACK] Seq=613 Ack=1 Win=65280 Len=13140 [TCP PDU reassembled in 171]
133	3.661816	10.45.7.218	128.119.245.12	TCP	27794	58306 → 80	[PSH, ACK] Seq=13753 Ack=1 Win=65280 Len=27740 [TCP PDU reassembled in 171]
146	4.009082	10.45.7.218	128.119.245.12	TCP	29254	58306 → 80	[PSH, ACK] Seq=41493 Ack=1 Win=65280 Len=29200 [TCP PDU reassembled in 171]
149	4.009388	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[ACK] Seq=70693 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
152	4.009452	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[PSH, ACK] Seq=82373 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
154	4.010026	10.45.7.218	128.119.245.12	TCP	2974	58306 → 80	[ACK] Seq=94053 Ack=1 Win=65280 Len=2920 [TCP PDU reassembled in 171]
162	4.357201	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=96973 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]
166	4.357372	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=117413 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]

Frame 133:	Packet, 27794 bytes on wire (222352 bits), 27794 bytes captured (222352 bits) on interface \Device\NPF_{A579126C-970A-4ACF-A3D4-000000000000}	0020	f5 0c e3 c2 00 50 a1 2a
Ethernet II, Src:	Dell 60:14:59 (c8:4b:d6:60:14:59), Dst: HuaweiTechno_ba:bd:49 (80:69:33:ba:bd:49)	0030	00 ff 87 91 00 00 67 65
Internet Protocol Version 4, Src:	10.45.7.218, Dst: 128.119.245.12	0040	68 65 79 20 6d 75 73 74
Transmission Control Protocol, Src Port:	58306, Dst Port: 80, Seq: 13753, Ack: 1, Len: 27740	0050	68 65 20 63 61 72 72 69
Source Port:	58306	0060	20 74 68 6f 75 67 68 74
Destination Port:	80	0070	6f 77 20 66 75 6e 6e 79
[Stream index: 2]		0080	73 65 65 6d 2c 20 73 65
[Stream Packet Number: 12]		0090	65 73 65 6e 74 73 20 74
[Conversation completeness: Incomplete, DATA (15)]		00a0	6f 77 6e 20 66 65 65 74
[TCP Segment Len: 27740]		00b0	6f 77 20 6f 64 64 20 74
Sequence Number: 13753 (relative sequence number)		00c0	63 74 69 6f 6e 73 20 77
Sequence Number (raw): 2703896035		00d0	21 0d 0a 0d 0a 20 20 20
[Next Sequence Number: 41493 (relative sequence number)]		00e0	20 41 4c 49 43 45 27 53
		00f0	4f 4f 54 3c 20 45 53 51

133	3.661016	10.45.7.218	128.119.245.12	TCP	27794	58306 → 80	[PSH, ACK] Seq=13753 Ack=1 Win=65280 Len=27740 [TCP PDU reassembled in 171]
146	4.009082	10.45.7.218	128.119.245.12	TCP	29254	58306 → 80	[PSH, ACK] Seq=41493 Ack=1 Win=65280 Len=29200 [TCP PDU reassembled in 171]
149	4.009388	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[ACK] Seq=70693 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
152	4.009452	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[PSH, ACK] Seq=82373 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
154	4.010026	10.45.7.218	128.119.245.12	TCP	2974	58306 → 80	[ACK] Seq=94053 Ack=1 Win=65280 Len=2920 [TCP PDU reassembled in 171]
162	4.357201	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=96973 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]
166	4.357372	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=117413 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]

Frame 146:	Packet, 29254 bytes on wire (234032 bits), 29254 bytes captured (234032 bits) on interface \Device\NPF_{A579126C-970A-4ACF-A3D4-000000000000}	0020	f5 0c e3 c2 00 50 a1 2a
Ethernet II, Src:	Dell 60:14:59 (c8:4b:d6:60:14:59), Dst: HuaweiTechno_ba:bd:49 (80:69:33:ba:bd:49)	0030	00 ff 87 91 00 00 2e 27
Internet Protocol Version 4, Src:	10.45.7.218, Dst: 128.119.245.12	0040	41 6e 20 61 72 6d 2c 20
Transmission Control Protocol, Src Port:	58306, Dst Port: 80, Seq: 41493, Ack: 1, Len: 29200	0050	65 21 20 20 20 57 68 6f
Source Port:	58306	0060	77 20 6f 6e 65 20 74 68
Destination Port:	80	0070	20 20 57 68 79 2c 20 69
[Stream index: 2]		0080	20 74 68 65 20 77 68 6f
[Stream Packet Number: 18]		0090	77 21 27 0d 0a 0d 0a 20
[Conversation completeness: Incomplete, DATA (15)]		00a0	69 74 20 64 6f 65 73 2c
[TCP Segment Len: 29200]		00b0	6f 75 72 3a 20 20 62 75
Sequence Number: 41493 (relative sequence number)		00c0	6e 20 61 72 6d 20 66 6f
		00d0	64 74 3c 20 45 53 51 4a

149	4.009388	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[ACK] Seq=70693 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
152	4.009452	10.45.7.218	128.119.245.12	TCP	11734	58306 → 80	[PSH, ACK] Seq=82373 Ack=1 Win=65280 Len=11680 [TCP PDU reassembled in 171]
154	4.010026	10.45.7.218	128.119.245.12	TCP	2974	58306 → 80	[ACK] Seq=94053 Ack=1 Win=65280 Len=2920 [TCP PDU reassembled in 171]
162	4.357201	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=96973 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]
166	4.357372	10.45.7.218	128.119.245.12	TCP	20494	58306 → 80	[PSH, ACK] Seq=117413 Ack=1 Win=65280 Len=20440 [TCP PDU reassembled in 171]

Frame 152:	Packet, 11734 bytes on wire (93872 bits), 11734 bytes captured (93872 bits) on interface \Device\NPF_{A579126C-970A-4ACF-A3D4-000000000000}	0020	f5 0c e3 c2 00 50 a1 2a
Ethernet II, Src:	Dell 60:14:59 (c8:4b:d6:60:14:59), Dst: HuaweiTechno_ba:bd:49 (80:69:33:ba:bd:49)	0030	00 ff 87 91 00 00 55 55
Internet Protocol Version 4, Src:	10.45.7.218, Dst: 128.119.245.12	0040	20 41 6c 69 63 65 20 68
Transmission Control Protocol, Src Port:	58306, Dst Port: 80, Seq: 82373, Ack: 1, Len: 11680	0050	0a 20 20 54 68 65 20 68
Source Port:	58306	0060	6f 6f 6b 20 68 69 73
Destination Port:	80	0070	72 6e 66 75 6c 6c 75
[Stream index: 2]		0080	21 27 20 68 65 20 72
[Stream Packet Number: 24]		0090	60 57 65 20 71 75 61
[Conversation completeness: Incomplete, DATA (15)]		00a0	61 73 74 20 4d 61 72
[TCP Segment Len: 11680]		00b0	62 65 66 6f 72 65 20
Sequence Number: 82373 (relative sequence number)		00c0	61 64 2c 20 79 6f 75
		00d0	0a 20 20 66 66 6e 75

7. What is the minimum amount of available buffer space advertised to the client by gaia.cs.umass.edu among these first four data-carrying TCP segments⁷? Does the lack of receiver buffer space ever throttle the sender for these first four data-carrying segments?

Answer here:

- Minimum advertised receiver window = 65,280 bytes
- Receiver buffer ไม่ได้จำกัดการส่งข้อมูลของ sender ในช่วงแรกของการส่งข้อมูล

⁷ Give the Wireshark-reported value for “Window Size Value” which must then be multiplied by the Window Scaling Factor to give the actual number of buffer bytes available at gaia.cs.umass.edu for this connection.

47	2.454449	10.45.7.218	128.119.245.12	TCP	54 58306 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
115	3.312070	10.45.7.218	128.119.245.12	TCP	666 58306 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=612 [TCP PDU reassembled in 171]	
116	3.312263	10.45.7.218	128.119.245.12	TCP	13194 58306 → 80 [ACK] Seq=613 Ack=1 Win=65280 Len=13140 [TCP PDU reassembled in 171]	
133	3.661016	10.45.7.218	128.119.245.12	TCP	27794 58306 → 80 [PSH, ACK] Seq=13753 Ack=1 Win=65280 Len=27740 [TCP PDU reassembled in 171]	
146	4.009082	10.45.7.218	128.119.245.12	TCP	29254 58306 → 80 [PSH, ACK] Seq=41493 Ack=1 Win=65280 Len=29200 [TCP PDU reassembled in 171]	

Frame 47:	Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{A579126C-970A-4ACF-A3D4-1E8BE79F6B50}	0000	80 69 33 ba bd 49 c8 4
Ethernet II, Src:	Dell_60:14:59 (c8:4b:d6:60:14:59), Dst: HuaweiTechno_ba:bd:49 (80:69:33:ba:bd:49)	0010	00 28 7f b4 40 00 80 6
Internet Protocol Version 4, Src:	10.45.7.218, Dst: 128.119.245.12	0020	f5 0c e3 c2 00 50 a1 2
Transmission Control Protocol, Src Port:	58306, Dst Port: 80, Seq: 1, Ack: 1, Len: 0	0030	00 ff 87 a5 00 00

Source Port:	58306
Destination Port:	80
[Stream Index: 2]	
[Stream Packet Number: 3]	
[Conversation completeness: Incomplete, DATA (15)]	
[TCP Segment Len: 0]	
Sequence Number: 1 (relative sequence number)	
Sequence Number (raw): 2703882283	
[Next Sequence Number: 1 (relative sequence number)]	
Acknowledgment Number: 1 (relative ack number)	
Acknowledgment number (raw): 2167912117	
0101 = Header Length: 20 bytes (5)	
Flags: 0x010 (ACK)	
Window: 255	
[calculated window size: 65280]	
[Window size scaling factor: 256]	
Checksum: 0x87a5 [unverified]	
[Checksum Status: Unverified]	
Urgent Pointer: 0	
[Timestamps]	
[SEQ/ACK analysis]	
[Client Contiguous Streams: 1]	
[Server Contiguous Streams: 1]	

8. How much data does the receiver typically acknowledge in an ACK among the first ten data-carrying segments sent from the client to gaia.cs.umass.edu? Can you identify cases where the receiver is ACKing every other received segment (see Table 3.2 in the text) among these first ten data-carrying segments?

Answer here:

ผู้รับข้อมูลส่ง ACK แบบสะสม โดยยืนยันข้อมูลครั้งละ 1000-2000 ไบต์ และมีพฤติกรรม ACK ทุก ๆ ประมาณ 2 TCP segment (Delayed ACK)

46	2.454276	128.119.245.12	10.45.7.218	TCP	66 80 → 58306 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128	
88	2.906072	35.186.229.172	10.45.7.218	TCP	66 443 → 57983 [ACK] Seq=1 Ack=2 Win=997 Len=0 SLE=1 SRE=2	
127	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=613 Win=63744 Len=0	
128	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=6453 Win=75776 Len=0	
129	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=9373 Win=81664 Len=0	
130	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=10833 Win=81792 Len=0	
131	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=12293 Win=80384 Len=0	
132	3.660901	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=13753 Win=78976 Len=0	
141	4.008981	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=15213 Win=84736 Len=0	
142	4.008981	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=16673 Win=87680 Len=0	
143	4.008981	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=22513 Win=87424 Len=0	
144	4.008981	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=25433 Win=90240 Len=0	
145	4.008981	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=28353 Win=90240 Len=0	
147	4.009343	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=29813 Win=91648 Len=0	
148	4.009343	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=34193 Win=87424 Len=0	
150	4.009436	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=37113 Win=90240 Len=0	
151	4.009436	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=40033 Win=90240 Len=0	
153	4.009984	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=41493 Win=91648 Len=0	
158	4.357165	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=45873 Win=88832 Len=0	
159	4.357165	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=48793 Win=90240 Len=0	
160	4.357165	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=50253 Win=91648 Len=0	
161	4.357165	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=51713 Win=91648 Len=0	
163	4.357359	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=54633 Win=91648 Len=0	
164	4.357359	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=57553 Win=91648 Len=0	
165	4.357359	128.119.245.12	10.45.7.218	TCP	60 80 → 58306 [ACK] Seq=1 Ack=61933 Win=91648 Len=0	